# Python Engineering - Programming Theory Guide

CLASS 15 - Exceptions & Debugging | Mental model of failure

Robust programming is not about preventing every failure. It is about separating normal execution from abnormal conditions, preserving useful information, and deciding consciously where a failure should be handled.

## 1. Normal path vs exceptional path

```
NORMAL PATH
input -> validate -> compute -> output

EXCEPTIONAL PATH
input -> problem detected -> exception -> handler or termination
```

An exception is a structured way to leave the normal path when continuing no longer makes sense.

## 2. Error is not the same as bug

| Concept | Example | Meaning |
|---|---|---|
| Expected external failure | Missing file | Environment condition is not satisfied |
| Invalid input | NOT_A_NUMBER | Data violates contract |
| Bug | Wrong index / wrong algorithm | Our code is defective |
| Syntax error | Invalid Python grammar | Code cannot be parsed |

Not every exception should be 'solved' with except. Sometimes it reveals a bug that should be fixed.

## 3. Exception propagation

```
main()
 -> test_file()
    -> load_trajectory_csv()
       -> float('NOT_A_NUMBER')
          -> ValueError
             -> propagates upward until matched
```

This lets low-level code detect a problem and higher-level code decide the appropriate response.

## 4. Detect where you know; handle where you can decide

```
file_io.py detects:
- invalid header
- wrong columns
- invalid numeric conversion

main.py decides:
- report
- skip file
- continue
```

Detect where you have enough information to recognize the problem; handle where you have enough context to choose a response.

## 5. Specific exceptions preserve meaning

```
FileNotFoundError -> resource location problem
ValueError        -> resource exists but value/content is invalid
```

Collapsing both into 'ERROR' throws away useful diagnostic information.

## 6. Handling vs hiding

```
HANDLING
except ValueError as error:
    report_bad_data(error)
    continue_with_next_file()

HIDING
except:
    pass
```

## 7. Fail fast

If an essential data contract is broken, stopping that load early can be safer than producing plausible but wrong results.

## 8. Data contract

```
header = h_m,v_m_s,t_s
each row = exactly 3 columns
each field = convertible to float
```

Validating at the input boundary protects the rest of the program.

## 9. raise as a design tool

raise expresses our own program rules, not only failures originating inside Python libraries.

```
if len(values) != 3:
    raise ValueError(...)
```

## 10. Keep try small

Large try blocks make it harder to know which operation actually produced a captured exception.

```
try:
    trajectory = load_trajectory_csv(path)
except ValueError:
    ...
else:
    analyze(trajectory)
```

## 11. finally and resources

finally means 'run this no matter what'. For files, with open(...) is usually the clearer resource-management tool.

## 12. Debugging: observe state, do not guess

```
EXPECTATION
values[1] should be numeric text

OBSERVATION
values[1] == 'NOT_A_NUMBER'

CONCLUSION
float(values[1]) will fail
```

The core debugging skill is comparing expectation with observation.

## 13. Breakpoints as experiments

A breakpoint asks the program: 'when you get here, show me your state'.

## 14. Step Into vs Step Over

```
trajectory = load_trajectory_csv(path)

Step Over -> execute loader as one operation
Step Into -> enter loader and inspect it line by line
```

## 15. Call Stack

```
main
 -> test_file
    -> load_trajectory_csv
       -> current line
```

It answers: how did execution reach this point?

## 16. Traceback: post-failure debugging

```
bottom -> ValueError + message
above  -> failing line in file_io.py
above  -> caller in main.py
```

Start near the bottom, identify type/message, then move upward into your code.

## 17. Error messages as instrumentation

```
weak:   'Invalid data'
better: 'Line 4: expected 3 columns, got 2'
```

## 18. Why not return None for everything

A single None loses the reason for failure. Exceptions preserve structured failure information.

## 19. Robustness does not mean always continue

| Context | Possible response |
|---|---|
| Batch of 100 files; one is corrupt | Report and continue |
| Critical configuration missing | Stop application |
| Optional data missing | Use documented default |
| Unexpected programmer bug | Let it surface and fix code |

## 20. Engineering connection

| System | Expected failures |
|---|---|
| GNSS processing | missing epoch, malformed measurement file |
| Test bench | missing sensor log, non-numeric field |
| Trajectory simulation | invalid config, impossible parameter |
| Optimization | solver failure / invalid domain |
| Data pipeline | wrong columns, corrupt record |

## 21. Final mental model

```
normal execution
 -> invalid condition detected
 -> EXCEPTION OBJECT
    type + message + call context
 -> propagate upward
 -> matching handler: controlled response
    OR
 -> no handler: traceback + stop
```

**Debugging:** pause, observe, compare with expectation, follow the flow. **Exceptions:** preserve and communicate information when normal execution is no longer valid.

## Final summary

```
TRY      -> code that may fail in an expected way
EXCEPT   -> response to a specific exception
ELSE     -> success-only code
FINALLY  -> always-run code
RAISE    -> deliberately signal invalid state/input

GOOD PRACTICE
- specific exceptions
- small try blocks
- clear messages
- do not hide bugs
- validate data at boundaries

DEBUGGING LOOP
predict -> pause -> inspect -> compare -> isolate -> fix -> rerun
```